

Deep Research Report

zk-Circuit Security in Rust

A Comprehensive Analysis for the zpai Project

Security Analysis for Zero-Knowledge Virtual Machines

Rust-Based zkVM Internals and Circuit Security

February 2026

1. Executive Summary

This research report provides a comprehensive analysis of security vulnerabilities and best practices for zero-knowledge (ZK) circuits implemented in Rust, with specific focus on zkVM (zero-knowledge virtual machine) systems. The findings are particularly relevant for the zpai project, an open security analysis platform targeting Rust-based zk-circuits.

Zero-knowledge virtual machines represent a paradigm shift in how developers interact with zero-knowledge proof systems. By abstracting away the complex cryptographic details, zkVMs like SP1, RISC Zero, and OpenVM enable developers to write standard Rust code while generating cryptographic proofs of correct execution. However, this abstraction does not eliminate security risks; rather, it shifts them to different layers of the stack. Understanding these vulnerabilities is crucial for building secure zk-powered applications.

Key Findings

- Soundness vulnerabilities (where invalid execution can be proved correct) are the highest priority concerns, with several critical issues discovered in production zkVM systems during 2024-2025.
- Under-constrained circuits remain the most frequent vulnerability class, accounting for the majority of ZK-related security incidents.
- The SP1 zkVM has undergone multiple security audits revealing specific vulnerability patterns including chip ordering validation issues, recursive verifier constraints, and Fiat-Shamir transcript handling bugs.
- Field arithmetic issues, particularly those involving BabyBear and BN254 field interactions, pose unique challenges for Rust zkVM implementations.
- The Plonky3 dependency layer has introduced soundness vulnerabilities that propagated to dependent projects like SP1, highlighting supply-chain security concerns.

2. ZK-Circuit Security Fundamentals

2.1 Soundness vs. Completeness

Zero-knowledge proof systems must satisfy two fundamental security properties: soundness and completeness. Understanding the distinction between these properties is essential for security analysis. Soundness ensures that a malicious prover cannot convince an honest verifier of a false statement. When soundness is broken, an attacker can generate proofs for invalid executions, which is catastrophic for systems relying on ZK proofs for integrity guarantees.

Completeness, on the other hand, ensures that an honest prover can always convince an honest verifier of a true statement. When completeness is broken, legitimate computations cannot be proven or verified, leading to denial of service scenarios. While completeness issues are serious, they typically have lower severity than

soundness issues because they do not allow attackers to forge invalid proofs.

Soundness Bug Severity Matrix

Likelihood \ Impact	High	Medium
High	Critical	High
Medium	High	Medium

Table 1: Severity assignment for soundness vulnerabilities

2.2 The zkVM Architecture

Modern zkVMs like SP1, RISC Zero, and OpenVM follow a similar architectural pattern. The system compiles high-level code (typically Rust) to a target instruction set architecture, usually RISC-V or MIPS. The compiled program is then executed to generate an execution trace, which is proven using STARK (Scalable Transparent Arguments of Knowledge) proofs. These proofs are often wrapped into SNARKs for efficient on-chain verification.

The SP1 architecture, for example, uses a CPU-like design where every instruction execution is cryptographically proven. The pipeline consists of four stages: compilation to RISC-V ELF binaries, execution and STARK proof generation, proof wrapping into SNARKs, and mathematical verification. This architecture provides developer familiarity and performance while maintaining cryptographic guarantees.

2.3 Host vs. Guest Separation

A critical security concept in zkVM programming is the separation between host and guest code. The host program runs in a standard execution environment and orchestrates proof generation, but is not part of the cryptographic proof. The guest program runs inside the zkVM and its execution is cryptographically proven. This separation has profound security implications: anything the host does is outside the cryptographic guarantees, and a malicious actor controlling the host can provide arbitrary inputs to the guest program.

Crucially, both host and guest code only execute during proving. The verifier performs pure mathematical verification without executing any source code. This means that critical validation logic must be implemented in the guest program to be cryptographically proven. Logic placed in the host cannot be trusted, as it can be manipulated by a malicious prover.

3. Vulnerability Taxonomy

The zkSecurity team has developed a comprehensive taxonomy of ZK circuit vulnerabilities through their zkBugs database. This taxonomy categorizes vulnerabilities by both their nature and root cause, providing a framework for systematic security analysis. Understanding this taxonomy is essential for both security auditors and developers working with ZK systems.

3.1 Vulnerability Categories

Category	Description	Impact
Under-Constrained	Insufficient constraints allowing verifier to accept invalid proofs	Soundness breach
Over-Constrained	Extra constraints causing rejection of valid witnesses	Completeness breach
Computational Issues	Erroneous computation in circuit logic	Completeness issues
Fiat-Shamir Issues	Errors in transcript handling or challenge derivation	Soundness breach
Backend Issues	Bugs in proof-system backend or verifier implementation	Soundness breach

Table 2: ZK Circuit Vulnerability Categories

3.2 Root Cause Analysis

Understanding the root causes of vulnerabilities is crucial for effective detection and prevention. The most common root causes identified in the zkBugs database include arithmetic field issues, assigned-but-unconstrained variables, missing constraints, and incorrect Fiat-Shamir implementations.

- **Arithmetic Field Issues:** Arithmetic overflows and underflows in zk circuits pose unique challenges due to the absence of runtime checks. These issues can lead to multiple solutions being accepted, making circuits underconstrained. In Rust zkVMs, the interaction between different field sizes (e.g., BabyBear vs. BN254) creates additional complexity.
- **Assigned but Unconstrained:** A frequent issue in zk circuit design lies in distinguishing between assignments and constraints. Variables may be assigned values based on correct computations but lack corresponding constraints, leading the verifier to accept any value for these variables.
- **Missing Constraints:** A required relation is never constrained or only partially constrained, allowing multiple or arbitrary assignments that still pass verification. This is the most common vulnerability pattern in ZK circuits.
- **Fiat-Shamir Issues:** Incorrect instantiation or use of the Fiat-Shamir transform, such as failing to absorb inputs or using wrong domain separation, breaks challenge binding to the transcript. Recent vulnerabilities in Plonky3 highlight the severity of this class.
- **Missing Verifier Checks:** Verifier-side checks are absent or incorrect, enabling proofs to bypass intended validation. The SP1 chip ordering vulnerability exemplifies this pattern.

4. Case Studies: Real-World Vulnerabilities

This section examines specific vulnerabilities discovered in production zkVM systems, with particular focus on SP1 and related projects. These case studies illustrate the practical manifestation of the vulnerability patterns discussed earlier and provide concrete examples for detection strategies.

4.1 SP1 Security Update (January 2025)

In January 2025, Succinct Labs disclosed three critical vulnerabilities affecting SP1 V3. These vulnerabilities were discovered through collaborative efforts by multiple security research teams, including Aligned, LambdaClass, 3MI Labs, Lev Soukhanov, and Onur Kilic. All three issues were soundness vulnerabilities with varying impact levels.

Vulnerability 1: Missing Validation of Chip Ordering (Medium Impact)

In SP1's STARK prover/verifier system, the prover provides a HashMap called `chip_ordering` to let the verifier know which polynomial evaluation claims correspond to which chip. The verifier uses this information to fetch the index of chips with preprocessed columns. However, validation that this `chip_ordering` correctly provides these indices was missing. An attacker could use incorrect openings of preprocessed commitments in SP1 proofs, leading to a soundness issue.

The recursive verifier was not affected because all valid verifier programs are generated in advance and committed into a Merkle tree. The fix involved making the verifier check that the indexed chip's name equals the name of the chip in question. This vulnerability demonstrates the importance of validating all prover-provided data structures in the verification circuit.

Vulnerability 2: Missing Check in Recursive Verifier (High Impact)

In the recursion circuits, the `is_complete` boolean flag denotes whether the current recursive proof represents complete program execution. The correctness of this flag is constrained by the `assert_complete` function, but the call to this function was missing in parts of the recursive verifier, including the first layer of recursion. This allowed proofs of partial execution to be verified as proofs of complete execution.

The attack vector exploited the fact that the hash of committed values is constrained to equal the public value's `committed_value_digest` at the end of the program, but partial execution would not perform this check. Proving partial execution as complete execution allowed incorrect `committed_value_digest` to be proved, leading to soundness issues. This vulnerability affected both the Rust SDK for verifying compressed proofs and the on-chain verifier for deferred proofs.

Vulnerability 3: Plonky3 Fiat-Shamir Issue (High Impact)

In SP1's STARK verifier, polynomial evaluation claims were batched using a random linear combination from the Plonky3 library. However, individual evaluation claims were not observed into the Fiat-Shamir challenger before sampling the coefficient for the random linear combination. This allowed incorrect polynomial evaluation claims to be verified, enabling incorrect chip proofs to pass verification.

The fix involved observing all evaluation claims into the Fiat-Shamir challenger correctly before sampling the random coefficient. This vulnerability propagated from Plonky3 as a critical dependency, highlighting supply-chain security concerns in ZK proof systems. The Plonky3 team issued a security advisory (GHSA-vrmm-4mm5-38vm) for this issue.

4.2 Additional SP1 Vulnerability Patterns

BabyBear/BN254 Range Check Issue (SND-004)

The Gnark recursion circuit constrains arithmetic over BabyBear when the native field of the ZKP circuit is the BN254 scalar field. Proper implementation requires range checking BN254 values to be less than the BabyBear modulus. In versions prior to 1.2.0, functions like `InvF` and `InvE` did not perform this range check. This issue (GHSA-f77q-r5qm-w4m8) allows hint-derived BN254 inverse values to lack BabyBear range checks, creating a soundness vulnerability.

This vulnerability pattern is particularly insidious because it arises from the interaction between different field sizes in recursive proof systems. The BabyBear field (approximately 2^{31}) is much smaller than the BN254 scalar field, and values represented in BN254 may not be valid BabyBear elements without explicit range checking.

Recursion VM LOAD Opcode Underconstraint (SND-009)

In the recursion VM, the `LOAD` opcode was underconstrained where register `a` was not explicitly bound to the loaded memory value. This is an example of an 'assigned but unconstrained' vulnerability pattern. The value was correctly loaded and assigned during execution, but the constraint system did not enforce the relationship between the loaded value and the destination register, allowing a malicious prover to supply arbitrary values.

4.3 RISC Zero Missing Constraint Vulnerability

A notable vulnerability in RISC Zero's zkVM resulted in a \$50,000 bounty despite prior audits. The vulnerability involved a missing constraint that allowed incorrect traces to pass verification. This case demonstrates that even well-audited zkVM implementations can contain subtle constraint bugs, and systematic testing approaches like Arguzz and zkFuzz are essential for comprehensive security.

5. Detection Strategies

Effective detection of zkVM vulnerabilities requires a multi-layered approach combining static analysis, dynamic testing, and formal verification. This section outlines strategies for identifying the vulnerability patterns discussed earlier.

5.1 Static Analysis Approaches

Static analysis of Rust zkVM codebases can identify many common vulnerability patterns. The `zpai` project implements a scanner engine for Rust source trees with initial rule sets targeting soundness and completeness vulnerabilities. Key detection patterns include:

- **Unsafe Block Detection (SND-001):** Identify unsafe blocks in security-critical paths. Unsafe code bypasses Rust's safety guarantees and should be carefully reviewed in constraint logic.
- **Debug Assertion Detection (SND-002):** Find debug-only assertions in critical validation logic. These assertions are compiled out in release builds, potentially leaving security checks unenforced.
- **Ignored Result Detection (SND-003):** Detect explicitly ignored verification results. Patterns like ``let _ = verify(...)`` indicate potential soundness issues.
- **Panic/Unwrap Detection (CMP-001, CMP-002):** Identify panic-style aborts and `unwrap/expect` calls in critical proof paths that could lead to completeness issues.

5.2 Dynamic Testing and Fuzzing

Dynamic testing approaches complement static analysis by executing the system with adversarial inputs. Recent research has produced specialized tools for zkVM testing:

`zkFuzz` is a framework for effective fuzzing of zero-knowledge circuits that focuses on soundness and completeness vulnerabilities arising from developer mistakes in tying constraints to computation. It formulates typical ZK circuit vulnerabilities as the Trace-Constraint Consistency Test (TCCT), a unified model for detecting discrepancies between execution traces and constraint enforcement.

`Arguzz` is a tool for testing zkVMs specifically for soundness and completeness bugs. It has been used to discover vulnerabilities like the RISC Zero missing constraint issue that resulted in a \$50,000 bounty, demonstrating the critical need for systematic testing of zkVMs.

5.3 Formal Verification

Formal verification provides the strongest assurance for zkVM correctness. The `CertiPlonk` framework from Nethermind enables formal verification of `Plonky3` ZK circuits, proving correctness of zkVM-scale systems using the Lean proof assistant. This approach can mathematically prove the absence of entire classes of vulnerabilities, though it requires significant expertise and effort.

Consistency verification frameworks have been developed for zero-knowledge virtual machines, providing unified abstract semantics for tabular constraint systems and inductive equivalence-checking algorithms. These approaches can verify that the verifier implementation matches the prover's expectations across all possible execution paths.

5.4 Precision Controls for `zpai`

The zpai project implements precision controls to minimize false positives while maintaining high coverage of real vulnerabilities. These controls include per-rule confidence gating via configuration files and regex/path suppression rules for known safe patterns. The goal is to achieve a high/critical false-positive rate below 20% for pilot repositories.

6. Security Tools and Frameworks

The ZK security ecosystem has developed various tools and frameworks for vulnerability detection, analysis, and mitigation. This section surveys the current landscape of security tools relevant to Rust zkVM development.

6.1 zkBugs Knowledge Base

The zkBugs project from zkSecurity provides a comprehensive knowledge base of known ZK vulnerabilities. The database includes structured descriptions of vulnerability patterns, root causes, and real-world examples with source code. This resource is invaluable for both security auditors developing detection rules and developers seeking to avoid common pitfalls.

The GitHub repository (zksecurity/zkbugs) contains reproducible vulnerabilities with complete descriptions and source code access. This allows security researchers to study vulnerability patterns hands-on and develop more effective detection strategies.

6.2 Audit Reports and Resources

Multiple audit firms have published reports on zkVM systems. SP1 has undergone audits from Veridise, Cantina, KALOS, and Zellic. These reports provide detailed technical analysis of discovered issues and serve as educational resources for understanding vulnerability patterns. The KALOS audit of SP1 identified several critical issues in the recursion VM CPU constraints.

Audit Firm	Target	Key Findings
KALOS	SP1 Recursion	CPU memory load constraints, is_complete handling
Veridise	SP1 Circuits	Precompiled function circuits, constraint coverage
Hexens	RISC Zero STARK-to-SNARK	Division-by-zero handling, memory consistency
Zellic	SP1 RV32IM Compliance	RISC-V specification compliance verification
Least Authority	Plonky3	Dependency security, implementation correctness

Table 3: Key zkVM Audit Reports and Findings

6.3 Bug Bounty Programs

SP1 maintains a bug bounty program on Code4rena, providing incentives for security researchers to discover vulnerabilities. The program covers the SP1 zkVM proving system, including proof generation and verification. Bug bounty programs complement traditional audits by providing continuous security testing and incentivizing responsible disclosure.

7. Best Practices for Secure zkVM Development

Based on the vulnerability patterns and case studies analyzed, this section provides actionable recommendations for developing secure zkVM applications in Rust.

7.1 Guest Program Security

- **Validate All Inputs:** The host can provide arbitrary inputs to the guest program. All input data must be validated within the guest, including range checks for numerical values, length limits for collections, format validation for structured data, and business logic constraints.
- **Use Panics for Invalid States:** In guest code, panics are the optimal way to handle invalid conditions. They prevent generation of proofs for invalid computations and provide clear failure modes. Use `assert!`, `panic!`, or `unwrap()` to fail fast on invalid inputs.
- **Enable Overflow Checks:** Rust's default integer overflow behavior differs between debug and release modes. Enable overflow checks in release mode by adding `overflow-checks = true` to the guest program's Cargo.toml profile.release section.
- **Handle 32-bit Architecture:** SP1 uses 32-bit RISC-V, which can cause issues when porting from 64-bit systems. Be aware of usize differences, pointer arithmetic limitations, and potential integer overflow in calculations that work fine on 64-bit systems.
- **Review Dependencies:** Libraries written for general platforms may assume OS access or 64-bit architecture. Prefer small, well-understood crates and test dependencies compiled for the RISC-V target.

7.2 Constraint System Design

- **Distinguish Assignment from Constraints:** Always verify that assigned values have corresponding constraints. Just because a variable is computed correctly does not mean the constraint system enforces this computation.
- **Validate Prover-Provided Structures:** Any data structure provided by the prover must be validated in the constraint system. The `chip_ordering` vulnerability in SP1 demonstrates the consequences of trusting prover input.

- **Implement Complete Transcript Handling:** Ensure all values are observed into the Fiat-Shamir challenger before sampling challenge values. The Plonky3 vulnerability shows the severity of incomplete transcript absorption.
- **Handle All Execution Paths:** Ensure constraints cover all possible execution paths, including error cases and partial executions. The `is_complete` vulnerability in SP1 arose from missing constraints in certain code paths.

7.3 Verification Key Management

Each guest program has a unique verification key derived from its compiled binary. Critical security considerations include verifying that the correct verification key is used during validation, ensuring keys are constant or follow correct update procedures, and preventing key substitution attacks. The derivation of keys is deterministic for each program, and auditors should verify keys by regenerating them independently.

8. Target Repositories for Analysis

The `zpai` project targets specific Rust zkVM repositories for security analysis. These repositories represent the primary zkVM implementations in the Rust ecosystem and provide diverse attack surfaces for vulnerability research.

Repository	Description	Key Components
<code>succinctlabs/sp1</code>	RISC-V zkVM with STARK proofs	Recursion circuits, STARK verifier, precompiles
<code>openvm-org/openvm</code>	Modular zkVM framework	Extension system, custom instructions
<code>matter-labs/zksync-airbender</code>	zkSync proving system	Recursive proofs, circuit optimization
<code>0xPolygonHermes/zisk</code>	Polygon zkVM	Proof aggregation, verification
<code>brevis-network/pico</code>	Lightweight zkVM	Efficient proving, mobile support

Table 4: Target Repositories for `zpai` Analysis

9. Current Detection Rules (`zpai` v1)

The `zpai` project implements a set of detection rules targeting common vulnerability patterns in Rust zkVM codebases. These rules are organized by severity and bug class.

9.1 Soundness Rules

Rule ID	Description
SND-001	Unsafe block in security-critical path
SND-002	Debug-only assertion in critical validation logic
SND-003	Verification result explicitly ignored
SND-004	Hint-derived BN254 inverse missing BabyBear range check
SND-005	Unchecked chip_ordering mapping in STARK verifier
SND-006	Underconstrained is_complete handling in recursive verifier
SND-007	Missing vk_root equality binding in recursive verification
SND-008	Missing permutation cumulative-sum transcript absorption
SND-009	Recursion VM LOAD opcode underconstraint
SND-010	Recursion VM CPU rows underconstrained when is_real = 0

Table 5: Soundness Detection Rules

9.2 Completeness and Other Rules

Rule ID	Description
CMP-001	Panic-style abort in critical proof path
CMP-002	Unwrap/expect in critical proof path
OTH-001	Overflow-prone embedded read_vec_raw pointer growth check
OTH-002	Missing embedded allocator heap-start bound assertion

Table 6: Completeness and Other Detection Rules

10. Future Research Directions

The field of zkVM security is rapidly evolving, and several areas warrant further research and development. This section outlines key directions for advancing security analysis capabilities.

10.1 AI-Assisted Security Analysis

The zkCraft framework introduces prompt-guided LLM approaches for ZK vulnerability detection, mapping the search for vulnerability-causing edits to algebraic existence statements. This approach leverages large language models as zero-shot mutation pattern oracles for discovering novel vulnerability patterns. Integrating

AI-assisted analysis into static analysis pipelines could significantly improve detection coverage for complex vulnerability patterns.

10.2 Improved Fuzzing Techniques

Advanced fuzzing techniques specifically designed for ZK circuits show promise for discovering deep vulnerabilities. The Trace-Constraint Consistency Test (TCCT) model from zkFuzz provides a unified framework for systematic testing. Future work should explore coverage-guided fuzzing with ZK-specific metrics and automated test case generation for known vulnerability patterns.

10.3 Formal Verification Integration

Integrating formal verification tools like CertiPlonk into continuous integration pipelines could provide mathematical guarantees for critical circuit components. Research into more accessible formal verification methods that don't require deep expertise would broaden adoption.

10.4 Cross-Platform Analysis

As the zkVM ecosystem grows, developing cross-platform analysis tools that can work across different zkVM implementations (SP1, RISC Zero, OpenVM, etc.) will be valuable. Standardized vulnerability taxonomies and detection rule formats could facilitate knowledge sharing across different platforms.

11. Conclusion

Zero-knowledge virtual machines represent a transformative technology for bringing zero-knowledge proofs to mainstream development. However, the security of these systems depends on rigorous analysis of complex constraint systems, proof protocols, and implementation details. This research has documented the key vulnerability patterns, real-world case studies, and detection strategies relevant to Rust zkVM security.

The zpai project aims to address these challenges through systematic static analysis, precision controls for minimizing false positives, and integration of ground truth seeds from known vulnerabilities. The current rule set targeting soundness and completeness vulnerabilities provides a foundation for comprehensive security analysis of Rust zkVM codebases.

As the zkVM ecosystem continues to evolve, maintaining robust security analysis capabilities will be essential for the integrity of systems built on these technologies. The combination of static analysis, dynamic testing, formal verification, and AI-assisted approaches offers a promising path toward securing the next generation of zero-knowledge applications.

References

1. Succinct Labs. SP1 Security Update (January 2025). <https://blog.succinct.xyz/sp1-security-update-1-27-25>
2. zkSecurity. zkBugs Knowledge Base. <https://bugs.zksecurity.xyz/descriptions>
3. zkSecurity. zkVM Security: What Could Go Wrong? <https://blog.zksecurity.xyz/posts/zkvm-security>
4. Sigma Prime. SP1 and zkVMs: A Security Auditor's Guide. <https://blog.sigmaprime.io/sp1-zkvm-security-guide.html>
5. zkFuzz: Foundation and Framework for Effective Fuzzing of Zero-Knowledge Circuits. arXiv:2504.11961
6. Arguzz: Testing zkVMs for Soundness and Completeness Bugs. arXiv:2509.10819
7. Nethermind. CertiPlonk: Formally Verifying Zero-Knowledge Circuits. <https://nethermind.io>
8. GitHub Security Advisory GHSA-vrmm-4mm5-38vm. Plonky3 Polynomial Evaluation Claims Issue.
9. GitHub Security Advisory GHSA-f77q-r5qm-w4m8. SP1 Gnark Recursion Circuit Range Check Issue.
10. L2BEAT. SP1 ZK Catalog. <https://l2beat.com/zk-catalog/sp1>